OMNI-CIO V10

Personal Autonomous Investment Platform

Канонический Master Plan разработки от А до Я

Статус: OWNER-APPROVED / PROGRAM LOCKED / DEVELOPMENT NOT STARTED

Базовая версия: Omni-CIO v9.0.0 (завершена и защищена как reference/rollback)

Дата формирования плана: 05.08.2026

0. Управление документом и статус программы

• Источник текущей базовой архитектуры: accepted main репозитория Misha2304/omni-cio-gateway.

• V9 имеет статус COMPLETE; новый feature work требует отдельной owner-approved программы согласно AGENTS.md.

• V10 не переписывает финансовые engines без доказанной необходимости: Research, Credit, Risk, Strategy, Optimizer, Immutable Plans, Safe Execution, Watchtower и Decision Journal наследуются из V9.

• Этот документ является программой разработки, а не доказательством реализации функций. Любая capability считается реализованной только после кода, тестов, CI, security gate и требуемой owner acceptance.

Содержание программы

- 1\. Цель V10 и целевая архитектура
- 2\. Неизменяемые инженерные правила
- 3\. Модель прогресса и Definition of Done
- BLOCK A - Архитектура и защита V9
- BLOCK B - GitHub, зависимости и CI/CD
- BLOCK C - Lenovo Omni-CIO Node
- BLOCK D - V10 Core и данные
- BLOCK E - Omni-CIO Web Platform
- BLOCK F - Telegram и связь с Web
- BLOCK G - AI Copilot, Research Intelligence и GPT Migration
- BLOCK H - Guardian / Watchtower Loop
- BLOCK I - Personal Real Execution
- BLOCK J - Системная интеграция, hardening и release
- 14\. Сквозная стратегия тестирования
- 15\. Работа с ошибками и инцидентами
- 16\. Правила моей будущей работы через GitHub
- 17\. Что намеренно НЕ входит в V10
- 18\. Минимальное участие владельца
- 19\. Финальный Definition of Done V10
- 20\. Реестр внешних технических ориентиров и источников

1. Цель V10 и целевая архитектура

Главная цель: вынести Omni-CIO из обязательной зависимости от Custom GPT и превратить готовый V9 в самостоятельную персональную инвестиционную операционную систему, работающую 24/7 на постоянно включённом Lenovo, управляемую через GitHub, с единым сайтом, Telegram-оповещениями и собственным Omni-CIO AI Copilot / Agent Orchestrator. AI-provider является сменным (локальный open-weight/Ollama first), а финансовый Core обязан сохранять безопасную детерминированную работу и явный degraded mode при недоступности AI.

• GitHub - Control Plane и долговременная память разработки: код, архитектура, тесты, зависимости, releases, runbooks и recovery.

• Lenovo - основной Personal Omni-CIO Node 24/7: runtime, DB, Watchtower, Web, Telegram, Research worker и локальный AI при необходимости.

• Omni-CIO Core - единое финансовое ядро V9/V10, не зависящее от конкретного пользовательского интерфейса.

• Web - главный кабинет владельца: портфель, Watchtower, Research, Risk, Decisions, Plans, Execution, Journal, System Health.

• Telegram - тревожная система и быстрый вход на правильную страницу Web; не отдельный финансовый мозг.

• Omni-CIO AI Copilot / Agent Orchestrator - собственный conversational и orchestration layer внутри Web/Telegram, который заменяет обязательную зависимость от Custom GPT, использует только разрешённые tools Core и не получает прямого broker-write authority.

• Local/replaceable AI Provider - локальный open-weight provider через Ollama является первым кандидатом; provider может быть заменён без переписывания Core. Недоступность AI не ломает финансовые safety engines.

• GPT Knowledge Migration - Instructions, Knowledge, Actions contracts, полезные prompt/workflow patterns и recovery-конфигурация существующего GPT сохраняются, нормализуются и мигрируют в versioned System Policy + Knowledge/RAG + Tool Registry. Custom GPT не удаляется до отдельного parity/decommission gate.

• T-Invest - источник реального портфеля/рынка и отдельный защищённый execution contour для OWNER.

PRIVATE GITHUB

\|

v

LENOVO OMNI-CIO NODE 24/7

\|-- Core (Portfolio / Risk / Optimizer / Watchtower / Plans / Journal)

\|-- PostgreSQL / Local State

\|-- Research Collector / AI Copilot / Research Agent

\|-- Web API + Web App

\|-- Telegram Bot

\`-- T-Invest read + isolated OWNER execution

1.1 Account Independence / Deployment Portability — обязательный архитектурный инвариант

V10 не принадлежит конкретному ChatGPT-аккаунту, разработчику, IDE, website-builder, хостингу или одному физическому компьютеру. Инженерный источник истины — private GitHub repository; runtime является заменяемым Data Plane.

```
PRIVATE GITHUB REPOSITORY
  = engineering Control Plane / source of truth

  code / Web / API / Telegram / AI policy
  configuration schemas and safe defaults
  database migrations
  tests / CI / release manifests
  bootstrap / deployment / recovery runbooks
          |
          | reviewed release
```

v
APPROVED V10 RUNTIME NODE
= replaceable Data Plane

local secret store
broker credentials
private portfolio/database state
Decision Journal private payloads
runtime cache / machine-specific state

Обязательные правила:

• Web является обычным source-controlled приложением и не может иметь authoritative state только в UI-builder/hosting dashboard;

• другой авторизованный разработчик должен восстановить engineering context из GitHub и продолжить branch -> tests -> PR -> review -> deploy без доступа к исходному ChatGPT-аккаунту;

• Lenovo — первый Personal V10 node, но не архитектурный владелец системы; перенос на другой approved node не должен требовать переписывания Web/Core;

• GitHub хранит код, схемы, policies и способы provisioning, но не реальные broker/Telegram tokens, encryption keys, полные account IDs или приватные portfolio payloads;

• GitHub engineering access никогда не означает broker trading authority;

• production release обязан идентифицироваться V10 version + Git SHA;

• до V10=100% обязателен clean-room portability/recovery rehearsal: GitHub + documented bootstrap + authorized backup + отдельно восстановленные secrets -> рабочий новый node;

• существующий OMNI_CIO_V10_ACCOUNT_INDEPENDENCE_AMENDMENT.md после этой консолидации сохраняется как audit trail исходного owner-approved amendment, но не как отдельный конкурирующий source of truth.

2. Неизменяемые инженерные правила

1. V9 не уничтожается и остаётся защищённой reference/rollback версией до полного V10 acceptance.

2. Один репозиторий, один Omni-CIO Core, один основной Web App. Запрещено плодить параллельные competing stacks без amendment к программе.

3. Real READ token, Sandbox token и Real TRADING token всегда разделены. Trading token не является fallback для чтения и не попадает в GitHub.

4. AI не имеет прямого PostOrder/execute права. AI Copilot может вести диалог, запускать allowlisted read-only/analytical tools, собирать и объяснять research/evidence и оркестрировать безопасные аналитические workflows; финансовые ограничения, sizing, immutable plan approval и execution authority находятся в детерминированном Core.

5. Каждый Real plan immutable. Изменение хотя бы одной существенной части отменяет старое подтверждение.

6. Возможная отправка заявки при сетевой неопределённости не повторяется автоматически: только reconciliation.

7. submitted никогда не приравнивается к filled без брокерского доказательства.

8. Любая внешняя информация имеет provenance, published_at/retrieved_at, freshness и identity binding.

9. Любой компонент обязан fail closed, если не хватает данных для безопасного сильного вывода или Real action.

10. Новая технология выбирается не по принципу latest=best, а после official-doc review, security/license/maintenance/Windows compatibility и доказанного преимущества.

11. Public/multi-user архитектура может быть подготовлена на уровне моделей, но не активируется в V10 Personal.

12. GitHub является единственным инженерным source of truth для Web/API/AI/Telegram/configuration contracts/migrations/deployment/recovery; account-only hidden configuration запрещена.

13. Доступ разработчика к GitHub не выдаёт broker/Telegram/encryption secrets и не расширяет real-money authority.

14. External Web ingress запрещён до отдельного TLS + OWNER authentication + session + CSRF/origin/host + rate-limit security gate. Real approval surface не публикуется наружу раньше этого gate.

15. Private local data и secret store должны учитывать угрозу физической потери Lenovo/SSD: full-disk или эквивалентная at-rest защита, отдельное хранение recovery key и проверяемое восстановление.

16. Системное время является security/data dependency: Windows/NTP clock sync контролируется; canonical timestamps хранятся в UTC; опасный clock drift блокирует short-lived confirmations, Telegram freshness checks и Real execution.

17. Logging/privacy policy задаётся до production: allowlist диагностических полей, обязательное masking/omission private data, log rotation/retention, crash-dump policy и audit-log integrity.

18. Runtime/OS/model updates выполняются как контролируемые изменения: Windows, GitHub Runner, NVIDIA driver, PostgreSQL, Ollama и AI model artifacts проходят compatibility/security/eval gate и имеют rollback; auto-update не может тихо изменить real-money behavior.

3. Модель прогресса и Definition of Done

Блок	**Назначение**	**Вес**	**Статус при старте**
A	Архитектура и защита V9	5%	NOT STARTED
B	GitHub, зависимости и CI/CD	9%	NOT STARTED
C	Lenovo Node / инфраструктура	12%	NOT STARTED
D	V10 Core и данные	14%	NOT STARTED
E	Web Platform	14%	NOT STARTED
F	Telegram	8%	NOT STARTED
G	AI Copilot, Research Intelligence и GPT Migration	18%	NOT STARTED
H	Guardian / Watchtower Loop	8%	NOT STARTED
I	Personal Real Execution	7%	NOT STARTED
J	Hardening / Recovery / Release	5%	NOT STARTED

• Процент блока засчитывается только после Exit Gate, а не после появления файлов или UI.

• После каждого accepted merge обновляется knowledge/17_V10_CURRENT_PROGRAM_STATUS.md: completed WPs, текущий WP, blockers, next step, общий процент.

• Следующий WP нельзя начинать, если текущий Exit Gate имеет P0/P1 blocker.

• В одном PR не смешивать несвязанные WP. Integration PR допускается только в Block J или когда сам WP требует связать уже принятые interfaces.

BLOCK A - Архитектура и защита V9 (5%)

WP0 - V10 Program Lock и baseline freeze

Цель: Официально открыть отдельную программу V10, не превращая завершённый V9 master-plan в бесконечную очередь задач.

Результаты / deliverables

• Создать канонический V10 master-plan в GitHub и knowledge/17_V10_CURRENT_PROGRAM_STATUS.md.

• Создать V10 system architecture, security boundaries, data flow, threat model и recovery contract.

• Обновить AGENTS.md: V9 COMPLETE; V10 - единственная разрешённая feature-program после owner approval.

• Зафиксировать точный V9 accepted commit/release, recovery assets и tests, которые нельзя регрессировать.

• Ввести ADR-процесс для архитектурных решений V10 (решение, причины, альтернативы, последствия, rollback).

Обязательная проверка

• Re-run V9 Regression Shield на точном baseline перед первым feature change.

• Проверить, что V10 docs не меняют production runtime и Actions.

• Проверить отсутствие секретов/приватных broker payloads в новых документах.

Exit Gate

• V9 восстанавливается по существующей документации.

• Есть один owner-approved roadmap V10 и один source of truth по текущему прогрессу.

• Нет незаявленного второго Gateway, второго execution stack или дублирующего проекта.

Участие владельца: Только утвердить программу V10 отдельной командой. До отдельной команды владельца START V10 функциональная разработка остаётся запрещена, а прогресс остаётся 0%.

WP0 pre-development plan-acceptance gate

До первого feature-кода WP0 обязан подтвердить, что:

• этот Markdown и owner-readable PDF содержательно синхронизированы и имеют новый SHA-256 manifest;

• веса A-J везде одинаковы и суммируются ровно в 100%;

• canonical status path — только knowledge/17_V10_CURRENT_PROGRAM_STATUS.md;

• AI/GPT migration и Account Independence уже находятся внутри канонического Master Plan;

• OMNI_CIO_V10_PLAN_READINESS_AUDIT.md не имеет открытых planning blockers;

• accepted main содержит V10 Program Lock;

• exact-head Regression Shield/security зелёный;

• V10 всё ещё 0% и не получает новых runtime/trading прав от самого факта утверждения плана.

BLOCK B - GitHub, зависимости и CI/CD (9%)

WP1 - Repository Architecture и boundaries

Цель: Подготовить репозиторий к многолетней разработке одной платформы без хаотичного размножения каталогов и legacy-слоёв.

Результаты / deliverables

• Определить финальные каталоги apps/api, apps/web, apps/telegram, omni_cio/core-модули, ai, infrastructure, docs, knowledge, tests.

• Добавить repository-boundary tests: public/read-only code не может импортировать real execution authority.

• Перед любым move/delete выполнять reference/import/CI/runtime audit и сохранять compatibility wrappers до доказанной миграции.

• Задокументировать canonical ownership каждого файла/каталога и запрет duplicate sources of truth.

Обязательная проверка

• Repository structure guard в CI.

• Import graph и circular dependency checks для критических packages.

• V9 compatibility tests после каждого structural PR.

Exit Gate

• Новый разработчик/AI по AGENTS + REPOSITORY_STRUCTURE понимает, где находится каждый компонент и что является authority.

• Нет двух competing реализаций одного финансового engine.

Зависимости: WP0

WP1 mandatory portability additions

• Зафиксировать repository ownership Web/API/AI/Telegram/infrastructure/configuration artifacts.

• Запретить authoritative application settings, существующие только в личном account dashboard.

• Development/test environment должен воспроизводиться из repository documentation + dependency locks + safe local config examples.

• Hosting/domain replacement считается deployment/configuration change, а не созданием нового сайта.

WP2 - Toolchain и Dependency Baseline

Цель: Зафиксировать поддерживаемый стек и способ обновления зависимостей до начала массовой разработки.

Результаты / deliverables

• Python/FastAPI сохранить как основной backend, если compatibility audit не обнаружит блокер.

• Frontend baseline: React stable + TypeScript + Vite stable, без обязательного SSR/RSC на первом V10.

• Local persistence baseline: PostgreSQL supported stable branch; exact minor выбирается перед implementation gate.

• Зафиксировать Node.js LTS, Python minor, package managers и lock formats.

• Ввести dependency policy: direct constraints, generated locks, SBOM/manifest, license allow/deny list.

Обязательная проверка

• Clean install на Windows из пустой рабочей директории.

• Deterministic lock verification.

• Known CVE audit и license scan.

• Build backend + frontend на GitHub-hosted CI до подключения Lenovo runner.

Exit Gate

• Один документированный toolchain; reproducible clean build; нет unpinned critical dependency paths.

Зависимости: WP1

WP3 - V10 Regression Shield

Цель: Сделать CI основным барьером против внесения скрытых дефектов до runtime.

Результаты / deliverables

• Сохранить V9 quality/security gates и добавить frontend TypeScript/lint/unit/E2E.

• Добавить DB migration tests, API schema/contract tests, authority tests, backup/restore tests, Telegram signature tests и AI schema tests.

• Добавить Playwright smoke/E2E для критических Web workflows.

• Разделить fast PR checks и full exact-head regression/security gate.

Обязательная проверка

• Compile, Ruff, format, mypy, pytest/coverage.

• Frontend build/type/lint/tests.

• Dependency audit, SAST, secret scan.

• Critical money-path replay/idempotency tests.

Exit Gate

• Ни один significant PR не может быть merged при красном mandatory gate.

• CI failure нельзя обходить отключением проверки.

Зависимости: WP2

WP4 - Release, build provenance и deployment contract

Цель: Каждый установленный runtime должен однозначно соответствовать конкретному reviewed Git commit.

Результаты / deliverables

• Ввести V10 semantic release version, git SHA, build SHA256 manifest, dependency lock fingerprint, DB migration version.

• Создать signed/hashed release manifest и проверку перед локальным deployment.

• Запретить deploy произвольного unreviewed working tree на OWNER runtime.

• Сформировать changelog/release notes автоматически из accepted PR metadata где практично.

Обязательная проверка

• Попытка deploy с неправильным manifest/hash должна fail closed.

• Rollback на предыдущий accepted release должен быть воспроизводим.

Exit Gate

• Можно доказать, какой commit и какие locks работают на Lenovo в любой момент.

Зависимости: WP3

WP4 mandatory release portability additions

• Accepted deploy выполняется только из reviewed Git commit/release evidence, не из untracked working tree.

• Release manifest фиксирует backend/frontend build hash, dependency/SBOM identity, migration version и Git SHA.

• Runtime /system/Admin UI показывает только non-secret installed version/Git SHA/build timestamp.

• Provider-specific deployment state минимизируется и документируется, чтобы перенос Lenovo -> другой approved node не создавал новый проект.

BLOCK C - Lenovo Omni-CIO Node (12%)

WP5 - Hardware/OS Audit и эксплуатационный профиль

Цель: Получить фактическую, а не предполагаемую картину постоянного 24/7 узла.

Результаты / deliverables

• Зафиксировать CPU, RAM 16/32 GB, точную GPU (GTX 1660 Ti или RTX 2060), VRAM, SSD/free disk, Windows build, NVIDIA driver.

• Проверить wired network, reboot behaviour, power settings, battery conservation, idle/load temperatures.

• Определить допустимые thermal/resource thresholds и alerts.

Обязательная проверка

• 30-60 минут контролируемой CPU/GPU нагрузки без перегрева/thermal shutdown.

• Reboot test, network reconnect test, disk health check.

Exit Gate

• Создан OWNER_NODE_PROFILE без серийных номеров/персональных данных; аппаратные лимиты известны.

Участие владельца: Один раз выполнить диагностические команды/дать скриншоты через удалённый рабочий стол при необходимости.

WP5 mandatory 24/7 node hardening additions

Hardware/OS audit дополнительно обязан проверить и записать:

• фактическое состояние full-disk encryption (BitLocker where supported или owner-approved equivalent) и отдельный recovery-key custody plan;

• Windows Time/NTP health, UTC canonical time policy и допустимый clock-drift threshold;

• battery/AC behavior, restart after power loss, thermals under sustained AI load и safe shutdown behavior;

• SSD free-space threshold и low-disk fail-safe;

• network interface/firewall baseline и запрет случайного public listener;

• какая часть данных считается private-at-rest и какие процессы имеют к ней ACL.

Exit Gate WP5 дополнительно: сценарий физической потери выключенного диска не должен автоматически раскрывать broker secrets/private DB; clock drift выше принятого threshold детектируется и блокирует чувствительные операции.

WP6 - OS identities и privilege separation

Цель: Не допустить, чтобы GitHub workflow автоматически обладал теми же правами, что real-money runtime.

Результаты / deliverables

• Разделить owner-admin, GitHub runner identity и Omni-CIO runtime identity.

• Минимизировать filesystem/registry/service permissions каждой роли.

• Runner не получает read access к trading secret store.

• Execution service получает только необходимые broker/network permissions.

Обязательная проверка

• Negative permission tests: runner не читает trading secret; Web process не может менять runner config; AI process не может вызвать broker write напрямую.

Exit Gate

• Компрометация обычного CI job не означает автоматический доступ к real-money secret.

Зависимости: WP5

WP6 mandatory privilege additions

• omni-ci-runner не получает право читать TINVEST_TRADING_TOKEN/Telegram secret/private DB dumps.

• omni-cio-runtime запускается с минимальными локальными ACL и не является Windows administrator без доказанной необходимости.

• Deployment controller принимает только reviewed release identity и не исполняет произвольный PR payload в контексте trading secrets.

• GitHub collaborator role и runtime-secret provisioning являются двумя независимыми процедурами.

WP7 - One-Time Bootstrap

Цель: Свести ручную первоначальную настройку Lenovo к небольшому числу проверяемых действий владельца.

Результаты / deliverables

• Создать versioned bootstrap-v10.ps1 и dry-run/preflight режим.

• Проверять/устанавливать runtime prerequisites, service users, directories, DB, runner, logs и service definitions.

• Bootstrap должен быть повторяемым/idempotent: повторный запуск не ломает установленную систему.

• Секреты не передавать arguments/console history; bootstrap только создаёт безопасные места для их последующего ввода.

Обязательная проверка

• Test на чистой/виртуальной Windows-среде где возможно.

• Повторный bootstrap поверх установленного node.

• Rollback/cleanup procedure.

Exit Gate

• Владелец не редактирует десятки конфигурационных файлов вручную.

Зависимости: WP6

Участие владельца: Запустить bootstrap от нужной роли и подтвердить системные UAC действия, которые невозможно выполнить через GitHub.

WP8 - Local Secret Store и rotation

Цель: Хранить broker/Telegram secrets вне GitHub и вне доступности обычных workflows.

Результаты / deliverables

• Разработать локальный Windows secret store на DPAPI/credential-protected подходе или выбранном local vault.

• Разделить read broker, trading broker, Telegram, local encryption keys и будущие external source credentials.

• Ввести secret version/rotation metadata без записи значений.

• Logs/errors всегда redacted.

Обязательная проверка

• Secret scan filesystem/Git history.

• Rotation rehearsal read token и Telegram token.

• Unauthorized process access test.

Exit Gate

• Нет plain-text secret в repo, logs, crash dumps или Web UI; runtime может получить только нужный ему секрет.

Зависимости: WP7

Участие владельца: Один раз локально ввести секреты, когда соответствующий контур будет готов.

WP8 mandatory secret lifecycle additions

• Для каждого секрета определить owner, purpose, scope, rotation/revocation procedure и признак компрометации.

• Recovery keys/private encryption material хранятся отдельно от GitHub и отдельно от зашифрованного backup, который они защищают.

• Никакой diagnostic bundle/crash dump не может включать raw secrets.

WP9 - Autostart, supervision и self-healing

Цель: После Windows Update/reboot система должна самостоятельно вернуться в рабочее безопасное состояние.

Результаты / deliverables

• Запуск DB, Core API, Web, Telegram, Watchtower, AI worker и health supervisor как контролируемых services/tasks.

• Dependency-aware startup order и health readiness.

• Crash restart policy с backoff и max attempts; запрет бесконечного restart storm.

• Отдельный degraded mode: AI/Telegram могут быть down, но Core/read-only state остаётся безопасным.

Обязательная проверка

• Kill каждого процесса и наблюдение восстановления.

• Полный reboot test без ручного открытия терминалов.

• Service dependency failure test.

Exit Gate

• После reboot OWNER получает HEALTHY/DEGRADED status без ручного запуска приложений.

Зависимости: WP8

WP9A - Runtime patching, time, observability и controlled restart policy

До 24/7 эксплуатации создать единый operations contract для:

• Windows security updates и predictable maintenance/reboot window;

• GitHub Runner update health;

• NVIDIA driver updates;

• PostgreSQL minor/security updates;

• Ollama runtime/model updates;

• service restart backoff, crash-loop detection и owner alert;

• NTP/time-drift monitoring;

• CPU/RAM/GPU/VRAM/temperature/disk thresholds;

• structured logging с private-data allowlist, rotation/retention и crash-dump policy.

Любой update, способный изменить calculations, AI behavior, DB compatibility или broker execution path, сначала проходит staging/regression/eval и имеет documented rollback.

WP10 - Backup, restore и off-device recovery

Цель: Не потерять решения, планы и audit trail при отказе SSD/Windows.

Результаты / deliverables

• Автоматические DB backups с retention и integrity hashes.

• Шифрованная off-device копия: отдельный диск/NAS/другое owner-controlled хранилище.

• Backup приватных non-secret конфигураций и model/RAG metadata.

• Runbook полного восстановления на новый Windows node.

Обязательная проверка

• Автоматический restore test из выбранного backup.

• Corrupt backup detection.

• Проверка, что secrets не попали в нешифрованный backup.

Exit Gate

• Restore доказан практикой, а не только написан в документе.

Зависимости: WP9

Участие владельца: Предоставить одно off-device место хранения или внешний диск, если его нет.

WP10 mandatory recovery objectives

WP10 обязан определить измеримые значения:

• RPO (Recovery Point Objective) — максимально допустимую потерю private durable state;

• RTO (Recovery Time Objective) — максимально допустимое время восстановления Personal V10 node;

• backup frequency/retention/off-device policy, выведенные из этих целей;

• отдельные recovery paths для GitHub-controlled source, encrypted DB/private state и runtime secrets.

Точные числа выбираются после WP5 hardware/storage audit, но отсутствие определённых и протестированных RPO/RTO является blocker для Exit Gate WP10.

BLOCK D - V10 Core и данные (14%)

WP11 - V9 Core Extraction

Цель: Отделить финансовые engines от обязательного Custom GPT presentation layer, сохранив проверенное поведение V9.

Результаты / deliverables

• Инвентаризировать все WP1-WP14 engines и их реальные зависимости от app/GPT wrappers.

• Создать стабильные Python service interfaces/DTO для Core.

• Сохранить legacy compatibility до завершения параллельных regression comparisons.

• Убрать UI/GPT assumptions из внутренних engines, не меняя финансовые формулы без отдельного proof.

Обязательная проверка

• Golden V9 fixtures: одинаковый input даёт эквивалентный output до/после extraction.

• No-GPT integration test запускает portfolio->risk->optimizer->plan.

Exit Gate

• Core может работать без Custom GPT и без OpenAI.

• V9 rollback остаётся рабочим.

Зависимости: Blocks A-C

WP12 - Canonical Domain Model и tenant-ready IDs

Цель: Создать единую внутреннюю модель данных, пригодную для OWNER сейчас и возможного multi-tenant будущего без включения public режима.

Результаты / deliverables

• Определить Tenant/User/BrokerAccount/PortfolioSnapshot/Position/Instrument/Policy/Evidence/Incident/Decision/Plan/Execution/Journal models.

• Каждый private record связывается с tenant_id/portfolio_id/policy_id.

• В V10 runtime разрешён только OWNER tenant; попытка создать public tenant fail closed.

• Identity instruments связывать по broker UID/ISIN/FIGI/class code, а ticker/name использовать только как secondary labels.

Обязательная проверка

• Schema validation, uniqueness/identity collision tests, cross-tenant access negative tests.

Exit Gate

• Нет неявных глобальных singleton-данных, которые сделали бы будущую изоляцию невозможной.

Зависимости: WP11

WP12 mandatory temporal/data identity additions

• Все persisted timestamps имеют явную timezone semantics; canonical storage — UTC, display timezone — presentation concern.

• Market/evidence timestamps различают published_at, retrieved_at, broker/server time и local receive time.

• Instrument/account/portfolio identities versioned; никакая AI/news строка не связывается с позицией только по похожему имени.

WP13 - PostgreSQL persistence и migrations

Цель: Перевести durable state из ad-hoc storage в версионированную локальную базу с auditability.

Результаты / deliverables

• Создать migration framework и схемы portfolio snapshots, positions, evidence, research, incidents, decisions, plans, execution, journal, jobs, health.

• Все деньги/цены/количества - Decimal/numeric с явной scale policy, не float.

• Append-only/audit-sensitive записи защищать от тихого overwrite.

• Indexes/retention для временных market/research caches.

Обязательная проверка

• Migration up/down или forward-recovery tests.

• Concurrent writes/idempotency tests.

• Backup/restore интеграция.

Exit Gate

• DB можно поднять из нуля миграциями и восстановить из backup; state hashes/revisions согласованы.

Зависимости: WP12

WP14 - Durable Job System

Цель: Сделать фоновые задачи наблюдаемыми, idempotent и восстанавливаемыми после перезапуска.

Результаты / deliverables

• Job model: id, type, idempotency key, state, lease, attempts, next_run, error class, correlation id.

• Задачи: portfolio refresh, market refresh, research, Watchtower, AI research, reconciliation, backups, health.

• Bounded retries; permanent/validation errors не повторять бесконечно.

• Job journal позволяет объяснить, что реально запускалось на сервере.

Обязательная проверка

• Crash mid-job recovery.

• Duplicate scheduling/idempotency.

• Lease expiry/concurrency.

Exit Gate

• После reboot незавершённые задачи либо корректно продолжаются, либо безопасно помечаются для reconciliation.

Зависимости: WP13

WP15 - T-Invest read-only data layer и freshness

Цель: Обеспечить канонические реальные данные портфеля для всех последующих решений.

Результаты / deliverables

• Адаптер account-specific read-only token.

• Portfolio/positions/market/operations streams или bounded polling по официальным возможностям API.

• Единые timestamps, freshness policy и data completeness markers.

• Кэш instrument identities; rate-limit aware backoff.

• Никаких торговых методов в read-only adapter.

Обязательная проверка

• Offline fixtures + controlled live read-only smoke.

• Stale/partial/stream reconnect tests.

• Wrong account/identity binding tests.

Exit Gate

• Core получает точный PortfolioSnapshot с completeness/freshness и masked account identity; real write невозможен из read adapter.

Зависимости: WP14

Участие владельца: Локально предоставить read-only T-Invest token, когда read layer будет готов.

BLOCK E - Omni-CIO Web Platform (14%)

WP16 - Web Design System

Цель: Создать один переиспользуемый UI-фундамент вместо десятков несогласованных страниц.

Результаты / deliverables

• Tokens: typography, spacing, semantic status, risk severity, light/dark.

• Components: cards, tables, charts wrappers, dialogs, drawers, buttons, badges, toasts, skeletons, empty/error states.

• Mobile-first от 360px; keyboard navigation; accessibility labels; reduced-motion support.

• Финансовые warning/error стили отделены от декоративных цветов.

Обязательная проверка

• Visual regression/screenshots ключевых components.

• Accessibility smoke.

• 360/768/1440 layout matrix.

Exit Gate

• Все последующие страницы используют единый design system; нет локальных случайных UI паттернов.

Зависимости: Block D core API contracts

WP17 - Application Shell и navigation

Цель: Сделать стабильный каркас единого сайта.

Результаты / deliverables

• Навигация: Обзор, Портфель, Активы, Watchtower, Research, Риски, Решения, Планы, Исполнение, Журнал, Система, Настройки.

• OWNER authentication/session; route guards; API error boundary.

• Global health/freshness indicator.

• Web build version/git SHA visible in system page, не в каждом экране.

Обязательная проверка

• Route/E2E tests, unauthorized access tests, offline API behaviour.

Exit Gate

• Сайт стабильно открывается локально и показывает системный shell даже при degraded downstream services.

Зависимости: WP16

WP18 - Portfolio Dashboard и Assets

Цель: Дать владельцу прозрачное текущее состояние капитала без обращения к GPT.

Результаты / deliverables

• Portfolio value, cash, positions, weights, currencies, issuers, sectors/classes, data timestamp.

• Concentration/top positions, stale data warnings, broker connection state.

• Position detail с instrument identity и linked incidents/research.

• Объяснимые расчёты и distinction broker/current market where applicable.

Обязательная проверка

• Golden portfolio UI fixtures, decimal/currency tests, empty portfolio, partial data, stale data.

Exit Gate

• Все показанные числа имеют источник/timestamp; UI не маскирует неполноту данных.

Зависимости: WP17, WP15

WP19 - Watchtower / Incident UI

Цель: Сделать каждую тревогу проверяемым объектом, а не transient сообщением.

Результаты / deliverables

• Incident timeline, severity, subject, evidence refs, current state, risk impact, dedup/reopen status.

• Переходы lifecycle и связь с Decision/Plan.

• Фильтры active/resolved/critical.

Обязательная проверка

• Incident state E2E; duplicate/reopened scenario; missing evidence display.

Exit Gate

• По каждому alert можно понять что произошло, почему, когда и что система сделала после обнаружения.

Зависимости: WP18

WP20 - Research Dossier UI

Цель: Сделать provenance и неопределённость видимыми владельцу.

Результаты / deliverables

• Facts, source links/labels, published/retrieved time, freshness, contradictions, unknowns, confidence.

• Event timeline; issuer/security identity; Credit/Fund links.

• AI generated summary явно отделяется от canonical structured facts.

Обязательная проверка

• Long text/source list rendering; stale/contradictory states; malicious markup sanitization.

Exit Gate

• Нельзя визуально спутать факт из источника с AI/inference.

Зависимости: WP19, Block G contracts

WP21 - Decision, Risk и Plan UI

Цель: Показать одно основное решение и точную математику до/после.

Результаты / deliverables

• MAIN_DECISION: NO_ACTION/HOLD/REDUCE/EXIT/REALLOCATE.

• Risk before/after, target weights, lots, costs, cash, constraints, blockers, evidence.

• Immutable plan hash/version/expiry.

• Альтернативы доступны как explanation, но не заставляют владельца угадывать правильную долю.

Обязательная проверка

• Deterministic rendering from Plan DTO; altered plan invalidates approval UI.

Exit Gate

• Владелец видит точный предмет будущего подтверждения до любого real execution.

Зависимости: WP20

WP22 - System/Admin UI

Цель: Дать владельцу единое место эксплуатации без командной строки.

Результаты / deliverables

• Service health, Git SHA/release, last deploy, DB/migrations, last backup, T-Invest, Telegram, Watchtower, AI, resource/temperature/free disk.

• Kill-switch state отображается read-only/controlled admin flow.

• Recent errors/jobs без secret leakage.

Обязательная проверка

• Degraded service scenarios; masked error output.

Exit Gate

• Владелец может понять состояние всей системы без GitHub/PowerShell для обычной диагностики.

Зависимости: WP21

WP22A - External Web Access / OWNER Authentication Security Gate

Цель: Не превращать локальный Personal кабинет в публично доступную поверхность до завершения security design.

Обязательные deliverables

• Решить, нужен ли вообще public ingress; безопасный private remote-access вариант рассматривается первым.

• Если внешний ingress нужен: TLS termination, certificate issuance/renewal, HSTS/host allowlist, trusted reverse-proxy chain.

• Отдельная OWNER authentication для ordinary browser; Telegram Mini App identity не является единственным способом входа.

• Server-side sessions: short/defined lifetime, rotation, explicit revoke/logout, secure/HttpOnly/SameSite cookies или эквивалентный reviewed механизм.

• CSRF protection, Origin/Referer/Host checks где применимо, CORS deny-by-default.

• Login/API rate limiting и lockout/abuse policy без возможности self-DoS критического monitoring path.

• Real-plan approval endpoint требует step-up/recent OWNER authentication и никогда не доверяет только deep-link URL.

• External ingress configuration repository-controlled; secret keys/cert private material остаются runtime-only.

Обязательная проверка

• ASVS 5.0.0 relevant controls; invalid/stolen/expired session; CSRF; host-header; origin; brute-force/rate-limit; direct API call without UI; Telegram initData replay/staleness.

Exit Gate

• До green gate Web остаётся local/private; Real approval surface не публикуется наружу.

BLOCK F - Telegram и связь с Web (8%)

WP23 - Telegram Bot Foundation

Цель: Создать отдельный лёгкий adapter к Core для уведомлений и быстрых owner-команд.

Результаты / deliverables

• OWNER-only bot; команды status, portfolio summary, active alerts, open app.

• Long polling на Personal Node на первом этапе; публичный webhook не требуется.

• Bot не содержит финансовой бизнес-логики - только Core API calls и presentation.

Обязательная проверка

• Unauthorized Telegram user denied; token masking; reconnect/backoff tests.

Exit Gate

• OWNER получает проверенный status/alerts, а чужой user ID не получает private data.

Участие владельца: Создать BotFather bot и один раз локально сохранить token.

WP24 - Telegram Identity и Mini App authentication

Цель: Криптографически связывать Web/Mini App session с OWNER.

Результаты / deliverables

• Validate Telegram initData server-side; не доверять initDataUnsafe.

• Проверять auth_date/expiry; OWNER whitelist; session rotation.

• Подготовить возможность future OIDC/third-party validation без изменения Core identity.

Обязательная проверка

• Tampered hash, replayed stale auth_date, wrong user ID tests.

Exit Gate

• Ни один Mini App request не получает OWNER scope только из client-provided fields.

Зависимости: WP23

Telegram authentication invariant

• Backend принимает Telegram identity только из валидированного Telegram.WebApp.initData; initDataUnsafe не является trusted authority.

• auth_date freshness проверяется server-side; replay/expired Mini App sessions отвергаются.

• Telegram OWNER ID whitelist не заменяет Web session security и не выдаёт broker secret.

WP25 - Alert Delivery Policy

Цель: Присылать только полезные material alerts и не превращать Watchtower в шум.

Результаты / deliverables

• Critical/Important/Info/Silent policy.

• Semantic dedup, reopen after resolution, quiet hours policy только для non-critical.

• Message содержит минимум private data, incident id и safe summary.

Обязательная проверка

• Duplicate delivery, persistent condition, resolved/reappeared condition.

Exit Gate

• Чистый Watchtower run молчит; новый material incident доставляется один раз.

Зависимости: WP24

WP26 - Telegram -> Web / Mini App integration

Цель: Использовать Telegram как точку входа в единый сайт, а не писать второй сложный UI.

Результаты / deliverables

• Кнопка открывает конкретный incident/decision route.

• Deep-link/start parameter validation.

• Поддержать ordinary browser и Telegram Mini App одним frontend codebase.

• External HTTPS ingress активируется только после отдельного security gate.

Обязательная проверка

• Deep-link wrong/expired incident; Telegram/mobile themes; safe-area/responsive.

Exit Gate

• Alert -> click -> правильный incident на Web без повторного ручного поиска.

Зависимости: WP25, Block E

BLOCK G - AI Copilot, Research Intelligence и GPT Migration (18%)

WP27 - Trusted Source Registry

Цель: Сначала создать данные и provenance; только потом добавлять AI.

Результаты / deliverables

• Иерархия: broker/exchange/regulator/issuer/official disclosure/rating agency/audited report; reliable media только secondary context.

• Для source: authority, allowed uses, legal/technical notes, parser type, freshness SLA.

• Запрет неизвестного источника формировать strong decision без explicit review.

Обязательная проверка

• Source allow/deny tests; timestamp/timezone normalization.

Exit Gate

• Каждый material claim может сослаться на canonical evidence record.

WP27 source-governance additions

Для каждого внешнего source registry хранит не только authority/freshness, но и:

• supported retrieval method/API/public page;

• terms/license/redistribution constraints, если применимы;

• parser version;

• expected update cadence;

• allowed retention of raw content vs normalized evidence.

Недоступный или юридически/технически запрещённый способ получения данных не обходится scraping-hack'ом без отдельного review.

WP28 - Research Collector и normalization

Цель: Автоматизировать сбор и дедупликацию внешних материалов без LLM-зависимости.

Результаты / deliverables

• Fetchers/parsers для утверждённых источников; caching/rate limits; content hash.

• Instrument/issuer identity resolution.

• Document normalization с безопасным удалением active content/HTML execution.

• Не хранить больше private/public raw data, чем нужно для evidence/replay policy.

Обязательная проверка

• Duplicate article, changed article, broken HTML/PDF, rate limit, timeout tests.

Exit Gate

• Один документ не порождает множество ложных событий; raw source provenance сохранён.

Зависимости: WP27

WP29 - Evidence/Claims Engine

Цель: Преобразовать документы в структурированный слой, который можно валидировать до использования в Risk.

Результаты / deliverables

• Claim types, supporting/contradicting evidence, materiality metadata, freshness, identity binding.

• Critical contradiction блокирует strong recommendation до разрешения/explicit uncertainty.

• LLM output никогда не становится evidence без source reference.

Обязательная проверка

• Contradiction cases, wrong instrument, stale source, unsupported claim.

Exit Gate

• Risk/Credit потребляют evidence contracts, а не свободный текст новости.

Зависимости: WP28

WP30 - AI Provider Abstraction

Цель: Сделать локальный AI сменным и необязательным.

Результаты / deliverables

• Provider interface: NONE и OLLAMA минимум.

• Timeout, cancellation, model metadata, prompt/schema version, deterministic-ish settings для classification where applicable.

• AI job isolated from trading authority и secret store.

Обязательная проверка

• Provider NONE full Core regression; provider down/deadlock; invalid response.

Exit Gate

• Отключение AI не ломает Watchtower/Core и не разрешает unsafe fallback.

Зависимости: WP29

WP30A - GPT Knowledge Migration и canonical AI knowledge source

Цель: Не потерять интеллект, правила и накопленные знания существующего Custom GPT при переходе на собственный AI layer.

Результаты / deliverables

• Инвентаризировать и version-control текущие GPT Instructions, Knowledge files, Actions/OpenAPI contracts, setup/recovery configuration и полезные workflow/prompt patterns.

• Разделить мигрируемый материал на: SYSTEM_POLICY (поведение/границы), KNOWLEDGE_RAG (справочные знания), TOOL_CONTRACTS (разрешённые действия), REFERENCE_GPT (архив/recovery).

• Не переносить устаревшие исторические инструкции как current authority; каждая запись получает source/version/status.

• Никакие секреты, реальные broker payloads или приватные conversation exports не коммитятся в GitHub.

Обязательная проверка

• Diff/inventory текущих GPT-visible assets против migration manifest; duplicate/conflicting instruction detection; secret scan.

Exit Gate

• Для каждой действующей функции GPT понятно, где находится её V10-эквивалент или почему она намеренно исключена; исходная GPT-конфигурация остаётся восстановимой.

Зависимости: WP30

WP30B - AI Knowledge/RAG и long-term memory architecture

Цель: Создать собственную управляемую память Omni-CIO вместо зависимости от Knowledge storage Custom GPT.

Результаты / deliverables

• Versioned Knowledge Base с provenance, document/version identity, freshness и controlled re-indexing.

• RAG retrieval policy: authority order, source filters, tenant/OWNER boundary, max-context budget, citation/source references.

• Разделить долговременную проектную память GitHub и приватное runtime state: portfolio/incident/research/decision/journal остаются в локальной DB и не попадают в GitHub.

• Conversation memory хранит только необходимый контекст и не становится источником истины для financial facts.

Обязательная проверка

• Wrong-document retrieval, stale knowledge, cross-scope leakage, poisoned document/prompt-injection retrieval, empty retrieval.

Exit Gate

• AI может объяснить ответ с указанием конкретного source/evidence, а отсутствие релевантного знания приводит к unknown/abstain, а не к выдумке.

Зависимости: WP30A

WP31 - Lenovo AI Benchmark и model selection

Цель: Выбрать минимальную модель, которая проходит именно Omni-CIO задачи на реальном железе.

Результаты / deliverables

• Проверить 4B/8B и при ресурсах 14B open-weight candidates через Ollama.

• Измерить tokens/sec, first-token latency, RAM/VRAM, GPU temperature, long context stability.

• Не фиксировать бренд навсегда; model registry хранит name/version/quantization/hash/eval score.

Обязательная проверка

• Одинаковый benchmark corpus; thermal soak; concurrent lightweight Core load.

Exit Gate

• Выбрана самая маленькая модель, удовлетворяющая установленным quality/latency порогам.

Зависимости: WP30

Участие владельца: Разрешить локальную установку Ollama/model weights; если место/железо не подходит, AI остаётся disabled.

WP31 mandatory model provenance / capacity additions

Каждый AI candidate фиксирует:

• exact model name/version/digest/quantization;

• license and source repository;

• expected RAM/VRAM/context limits;

• measured latency/throughput/thermal profile on Lenovo;

• eval-suite result and accepted threshold;

• rollback artifact/previous accepted model.

Model/runtime auto-update запрещён. Новая модель становится accepted только после повторного Omni-CIO eval и security/tool-authority checks.

WP32 - Structured AI Researcher

Цель: Использовать AI для чтения/синтеза, но не для прямого финансового исполнения.

Результаты / deliverables

• Strict JSON schema: event_type, facts, contradictions, unknowns, materiality, confidence, source refs.

• Tool calls только read-only research/core tools.

• Prompt injection defense: внешняя страница не может изменить system policy/tool authority.

• AI summary сохраняется отдельно от canonical evidence.

Обязательная проверка

• JSON validity; tool allowlist; malicious article prompt injection; hallucinated source; unsupported conclusion.

Exit Gate

• AI не может создать broker order или изменить risk limit; invalid output отбрасывается validator.

Зависимости: WP31

WP32A - AI Tool Registry и authority firewall

Цель: Дать собственному AI реальные возможности Omni-CIO без выдачи ему неконтролируемой власти.

Результаты / deliverables

• Единый versioned Tool Registry для Web AI, Telegram AI и autonomous research agent.

• Категории tools: read-only portfolio/market, research/evidence, analytical Core, plan preview; broker-write tools отсутствуют из AI registry.

• Per-tool input/output schema, timeout, audit id, authorization context, rate/concurrency limits.

• Tool output проходит validation/sanitization перед возвратом модели; model text не интерпретируется как executable instruction.

Обязательная проверка

• Unknown tool, argument injection, tool-chain escalation, excessive agency, repeated tool loop, malformed output.

Exit Gate

• Ни один AI prompt/документ/пользовательский текст не может расширить allowlist или получить broker-write/secret access.

Зависимости: WP32

WP32B - Omni-CIO AI Copilot / Agent Orchestrator

Цель: Создать собственный интеллектуальный интерфейс, который заменит Custom GPT как обязательный пользовательский слой.

Результаты / deliverables

• Один Agent Orchestrator для Web и Telegram: intent -> context -> allowed tools -> evidence -> answer/action proposal.

• Поддержать owner queries: portfolio status, asset investigation, Watchtower incident explanation, risk/credit questions, research request, optimizer/plan explanation и system status.

• Ответы обязаны отделять broker facts, deterministic Core calculations, source evidence, AI inference и unknowns.

• Агент не принимает owner approval за Real plan и не преобразует свободный текст в broker submit.

• Provider abstraction позволяет LOCAL/OLLAMA и будущие provider adapters; financial Core не зависит от конкретной модели.

Обязательная проверка

• Multi-turn context, stale session, ambiguous intent, prompt injection, tool-call loop, unsupported request, provider failure/degraded mode.

Exit Gate

• Владелец может управлять аналитическими функциями Omni-CIO на естественном языке без Custom GPT, при этом authority не шире детерминированных APIs.

Зависимости: WP32A

WP32C - Web AI Copilot и Telegram AI surface

Цель: Дать один и тот же AI-мозг в основном Web App и Telegram, не создавать два расходящихся бота.

Результаты / deliverables

• Web: отдельный Omni-CIO AI Copilot workspace с citations/evidence/tool activity summary и deep links на Portfolio/Incident/Research/Decision.

• Telegram: компактный conversational adapter к тому же Orchestrator; сложные результаты открываются в Web/Mini App.

• Единая session/identity/authorization модель; Telegram не хранит второй knowledge base или отдельные prompts.

• Degraded state явно сообщает, когда AI недоступен, а deterministic Portfolio/Watchtower функции продолжают работать.

Обязательная проверка

• Одинаковый intent даёт согласованный Core evidence в Web/Telegram; owner auth; long response handoff; session isolation.

Exit Gate

• Custom GPT больше не нужен как единственный способ разговаривать с Omni-CIO; Web и Telegram используют один V10 AI Orchestrator.

Зависимости: WP32B, Blocks E/F integration contracts

WP32D - Autonomous Research Agent

Цель: Использовать AI не только по запросу владельца, но и как ограниченного автономного исследователя material events.

Результаты / deliverables

• Watchtower/Research Collector может создавать AI research job только по утверждённым triggers/materiality policy.

• Agent собирает дополнительные trusted sources, сравнивает claims, ищет contradiction/noise и формирует structured research dossier.

• Бесконечный open-ended web crawl запрещён: source registry, query/time/token budgets, dedup и stop conditions обязательны.

• Autonomous agent не создаёт Real plan approval и не выполняет trades; результат возвращается в Evidence -> Credit/Risk -> Optimizer pipeline.

Обязательная проверка

• Noisy news burst, duplicate sources, malicious webpage instructions, missing official confirmation, resource budget exhaustion.

Exit Gate

• Material incident может получить автономный evidence-backed research dossier без участия владельца, но никакое финансовое действие не происходит только по решению LLM.

Зависимости: WP32C

AI/RAG security invariant

• External documents/news are untrusted data, never System Policy or tool authorization.

• Prompt-injection-shaped content cannot expand Tool Registry authority, reveal secrets or disable deterministic safety gates.

• Retrieved chunks carry provenance and are filtered/escaped before model use; model output remains untrusted until schema/evidence validation.

• AI context/logging avoids raw broker secrets/private identifiers; tool results expose least data required.

WP33 - AI Evaluation Harness и optional LoRA gate

Цель: Проверять не только модель, но весь AI Copilot/Research/Tool/RAG контур объективно до использования в реальном Guardian workflow.

Результаты / deliverables

• Corpus: downgrade/default/delisting/liquidity/report/news noise/duplicate/rumor/contradictions/stale/wrong identity.

• Metrics: grounded factual precision, materiality, contradiction detection, unknown recognition, schema success, tool discipline, retrieval quality, agent task success, excessive-agency violations и prompt-injection resistance.

• OWASP AI security requirements применить как security checklist для AI-enabled части.

• LoRA/QLoRA разрешить только после доказанного систематического gap; обучение модели с нуля вне scope.

Обязательная проверка

• Blind benchmark old/new model; adversarial/prompt injection suite; regression threshold.

Exit Gate

• AI provider допускается в production Guardian только после прохождения заранее заданных порогов; иначе AI_ENABLED=false.

Зависимости: WP32D

WP33A - Custom GPT parity, fallback и decommission gate

Цель: Не отключить существующий GPT раньше, чем собственный V10 AI доказал функциональный паритет по нужным возможностям.

Результаты / deliverables

• Parity matrix: текущие GPT Instructions/Knowledge/Actions/owner workflows -> V10 Copilot/RAG/Tools/Core equivalents.

• Golden сценарии сравнивают: portfolio Q&A, research, incident explanation, risk/optimizer explanation, missing-data behavior, source grounding и safety boundaries.

• Custom GPT остаётся reference/fallback до owner acceptance этого gate; его recovery/configuration assets не удаляются после decommission.

• Решение об отключении платной GPT-зависимости принимается только после доказанного V10 parity; до этого обе поверхности могут существовать параллельно без расхождения source of truth.

Обязательная проверка

• Blind parity suite + owner acceptance; проверить, что V10 не потерял критические знания/границы из существующего GPT.

Exit Gate

• Владелец может прекратить использовать Custom GPT без потери обязательных возможностей Omni-CIO; V10 AI остаётся provider-replaceable, а GPT config сохранён как audit/recovery reference.

Зависимости: WP33

BLOCK H - Guardian / Watchtower Loop (8%)

WP34 - Production Watchtower wiring

Цель: Превратить реализованный V9 Watchtower в реально подключённый V10 monitoring loop с canonical providers.

Результаты / deliverables

• Providers: portfolio, risk, credit, market/evidence, policy, data quality.

• Durable state в V10 DB, semantic fingerprints/dedup/reopen semantics.

• Watchtower не выполняет сделки.

Обязательная проверка

• Baseline first run; duplicate run; persistent event; resolved/reappeared; stale provider.

Exit Gate

• Периодическая проверка реального OWNER портфеля создаёт incident только на material change.

Зависимости: Blocks D,G

WP35 - Incident lifecycle engine

Цель: Управлять проблемой как полноценным state machine от обнаружения до закрытия.

Результаты / deliverables

• DETECTED -> INVESTIGATING -> EVIDENCE_READY -> RISK_RECALCULATED -> DECISION_READY -> OWNER_NOTIFIED -> RESOLVED/ESCALATED.

• Every transition journaled c correlation id.

• Retry/recovery rules per state.

Обязательная проверка

• Crash/restart в каждом состоянии; duplicate transition rejection.

Exit Gate

• Любой incident имеет однозначное текущее состояние и replayable history.

Зависимости: WP34

WP36 - Guardian Decision Pipeline

Цель: Связать Research, Credit/Risk, Strategy, Optimizer и Plans в один автономный analysis workflow.

Результаты / deliverables

• Watchtower trigger запускает research only when needed.

• Evidence validation до strong action.

• Optimizer рассчитывает target, lots, cash/risk before-after; AI не задаёт цифры напрямую.

• NO_ACTION/CASH остаётся полноценным результатом.

Обязательная проверка

• Golden incident scenarios; missing/contradictory evidence; optimizer infeasible result.

Exit Gate

• Для material incident формируется один Decision object с exact source inputs и reproducible output.

Зависимости: WP35

WP37 - One Main Decision policy

Цель: Не перекладывать профессиональный sizing на владельца через меню случайных процентов.

Результаты / deliverables

• MAIN_DECISION = NO_ACTION/HOLD/REDUCE/EXIT/REALLOCATE.

• Если confidence недостаточен: REVIEW_REQUIRED/INSUFFICIENT_EVIDENCE вместо ложной точности.

• Alternatives используются для объяснения decision dominance, а не как просьба владельцу угадать правильное действие.

Обязательная проверка

• Same inputs -> same decision; uncertainty threshold boundary tests.

Exit Gate

• Однозначное решение либо честный blocker; нет ложной "100% гарантии рынка".

Зависимости: WP36

WP38 - Explainability и owner alert package

Цель: Каждое решение должно быть доказуемым до отправки владельцу.

Результаты / deliverables

• Что произошло, что доказано, что неизвестно, impact на весь portfolio, target, before/after, costs, reasons, cancel triggers.

• Telegram получает concise alert; Web получает полный dossier.

• Every explanation links to Decision/Plan hashes.

Обязательная проверка

• No-source claim detection; numerical consistency UI/API.

Exit Gate

• Владелец может проверить почему система решила именно так, не доверяя "магии AI".

Зависимости: WP37

BLOCK I - Personal Real Execution (7%)

WP39 - Mandatory Shadow Mode

Цель: Проверить Guardian на реальном портфеле без права менять деньги.

Результаты / deliverables

• Все incidents/decisions/plans формируются как если бы execution существовал, но broker writes hard disabled.

• Journal сравнивает decision с NO_ACTION и фактическим последующим рынком, не выдавая hindsight как исходную истину.

• Собрать достаточное число реальных/синтетических событий и failure cases.

Обязательная проверка

• Несколько недель стабильной работы; zero unauthorized writes; alert quality review.

Exit Gate

• Owner принимает отдельное решение, что Shadow quality достаточна для открытия Real acceptance.

Зависимости: Blocks A-H

Участие владельца: Наблюдать реальные рекомендации и сообщать о явно неверных/неудобных результатах; никаких торговых подтверждений на этом этапе.

WP39 Shadow Mode acceptance criteria

Shadow duration is not a calendar checkbox. Exit requires enough representative incidents to exercise normal, stale-data, contradictory-research, AI-unavailable, broker-unavailable and no-action cases. If natural incidents are insufficient, deterministic replay/synthetic scenarios supplement, but do not falsify live evidence.

WP40 - Immutable Plan Approval

Цель: Одно подтверждение владельца разрешает только заранее полностью раскрытый plan.

Результаты / deliverables

• Plan hash связывает exact legs/assets/lots/price limits/costs/expiry/constraints.

• Approval short-lived, one-time, owner-bound; изменение plan invalidates approval.

• Web является основным экраном подтверждения; Telegram может только открыть plan/показать status.

Обязательная проверка

• Altered plan/replayed approval/expired approval/wrong owner tests.

Exit Gate

• Сервер не может выполнить другое действие под старым approval.

Зависимости: WP39

WP41 - Execution State Machine

Цель: Автоматически выполнить подтверждённый plan строго внутри его границ.

Результаты / deliverables

• PRE-FLIGHT -> SUBMIT LEG -> RECONCILE -> VERIFY -> NEXT LEG -> FINAL RECONCILIATION.

• Повторный market/portfolio check перед каждой ногой.

• Idempotency key per request/leg; one possible submit.

• Partial fill policy и cash recalculation.

Обязательная проверка

• Broker fake/integration cases: fill/partial/reject/timeout/possible-submit.

Exit Gate

• Никаких hidden extra legs; state machine воспроизводима по Journal.

Зависимости: WP40

Real execution additional preconditions

Real submit is additionally blocked when any of the following is true:

• system clock/time sync unhealthy beyond accepted threshold;

• OWNER session/confirmation expired or cannot be cryptographically bound to exact plan hash;

• durable state/DB is read-only, degraded or reconciliation state unknown;

• audit log cannot persist required immutable execution evidence;

• external ingress/auth gate is not green for the surface from which approval arrives.

WP42 - Drift Stop и re-plan

Цель: Не продолжать старое решение, если рынок/портфель ушёл за подтверждённые границы.

Результаты / deliverables

• Price/spread/liquidity/portfolio/policy drift thresholds.

• При breach - STOP, safe cash/position state, новый analysis/plan, новое approval.

• Запрет автоматического "догоняющего" расширения лимита.

Обязательная проверка

• Artificial price gap/spread widening/position change mid-plan.

Exit Gate

• Подтверждение означает permission to execute exact plan, а не бессрочную автономию.

Зависимости: WP41

WP43 - Broker reconciliation и ambiguous-submit safety

Цель: Сохранить ключевой safety invariant V9 в новой платформе.

Результаты / deliverables

• Broker state - единственное доказательство fill.

• Network timeout после possible submit -> UNKNOWN/RECONCILE, no automatic second submit.

• Order/portfolio streams reconcile execution journal.

Обязательная проверка

• Idempotent replay verified with no second submit.

• Submitted vs filled distinction.

Exit Gate

• Canonical V9 invariant сохраняется в V10 и покрыт permanent regression test.

Зависимости: WP42

WP44 - Kill switches и OWNER authority

Цель: Обеспечить быстрый отказ от опасного функционала независимо по контурам.

Результаты / deliverables

• REAL_TRADING_ENABLED, PLAN_EXECUTION_ENABLED, WATCHTOWER_ENABLED, AI_ENABLED, TELEGRAM_ENABLED.

• Kill switch state доступен System UI и service config; изменение требует owner/admin authority.

• AI/Telegram kill не должен ломать read-only portfolio monitoring.

Обязательная проверка

• Каждый switch on/off runtime tests.

Exit Gate

• Одним действием можно запретить реальные деньги без остановки аналитической системы.

Зависимости: WP43

BLOCK J - Системная интеграция, Hardening, Recovery и Release (5%)

WP45 - Full System Integration

Цель: Собрать уже завершённые блоки последовательно и доказать end-to-end workflow.

Результаты / deliverables

• GitHub -> reviewed release -> Lenovo -> DB -> T-Invest -> Watchtower -> Research/AI -> Risk -> Decision -> Telegram -> Web -> Plan -> Execution -> Journal.

• Все boundaries проверяются contract tests; не переписывать внутренние компоненты на этапе сборки без отдельного bug PR.

Обязательная проверка

• E2E shadow и controlled execution simulations.

Exit Gate

• Каждый переход имеет evidence; нет ручных скрытых шагов между сервисами.

Зависимости: Blocks A-I

WP46 - Fault Injection / Chaos для Personal Node

Цель: Искусственно сломать систему до эксплуатации и проверить, что отказ не приводит к неконтролируемой сделке.

Результаты / deliverables

• Отключение интернета, Telegram, AI, T-Invest, DB, disk-full simulation, stale market, duplicate messages, Windows reboot.

• Crash в разных execution states.

• Clock/time skew checks для TTL/approval.

Обязательная проверка

• Automated/controlled fault suite с expected safe state.

Exit Gate

• При каждом tested failure money boundary остаётся fail-closed/reconciled.

Зависимости: WP45

WP47 - Security Acceptance

Цель: Проверить Web/API/AI/local-node security как отдельный release gate.

Результаты / deliverables

• OWASP ASVS 5.0.0 как web/API security checklist; OWASP AISVS 1.0 для AI-enabled components.

• Auth/session/authorization, CSRF/XSS/injection, SSRF, file handling, secret exposure, local privilege, Telegram auth, backup encryption.

• SAST/dependency/secret scans и manual threat-model review.

Обязательная проверка

• Attack-path negative tests на owner data/trading authority.

Exit Gate

• Нет открытого P0/P1 security finding перед release.

Зависимости: WP46

WP47 mandatory expanded security acceptance

Security acceptance must explicitly cover:

• at-rest protection / physical disk-loss threat;

• OWNER browser auth, Telegram initData validation, session revocation and step-up approval;

• TLS/certificate/reverse-proxy configuration if external ingress enabled;

• CSRF/CORS/Origin/Host and external rate limits;

• GitHub runner -> deploy controller -> runtime privilege boundary;

• secret rotation/revocation and diagnostic/crash-dump leakage;

• logging allowlist, masking, retention and audit integrity;

• AI prompt injection, poisoned RAG, tool escalation/excessive agency and data exfiltration checks using OWASP AISVS as a verification baseline;

• dependency/SBOM/license findings and exact exceptions with owner-approved rationale.

WP48 - Performance/Thermal/24x7 Acceptance

Цель: Доказать, что выбранный Lenovo способен стабильно держать Personal V10.

Результаты / deliverables

• CPU/RAM/GPU/VRAM/disk/network baseline под обычной нагрузкой.

• AI heavy task не должен делать Web/Watchtower недоступными надолго; resource quotas/concurrency limits.

• Temperature monitoring и graceful AI throttling.

Обязательная проверка

• 24-72h soak test, research bursts, DB backup concurrent load.

Exit Gate

• Нет memory leak/thermal crash/resource starvation в acceptance window.

Зависимости: WP47

WP48 24/7 soak specifics

Soak test records CPU/RAM/GPU/VRAM, GPU/CPU temperatures, disk growth/free-space, DB size, queue depth, restart count, clock drift, Telegram/Web latency, AI latency and Watchtower cycle timing. Threshold breach must create a bounded alert rather than silently degrading into unsafe Real behavior.

WP49 - Disaster Recovery Rehearsal

Цель: Восстановить V10 как будто старый Lenovo/SSD потерян.

Результаты / deliverables

• Fresh/clean environment -> GitHub bootstrap -> DB restore -> local secrets restore -> health -> shadow verification.

• Runbook не зависит от памяти чата или устных инструкций.

Обязательная проверка

• Практический restore rehearsal с checklist/evidence.

Exit Gate

• Restore time и действия известны; V10 возвращается в read-only safe state до разрешения Real.

Зависимости: WP48

Участие владельца: Помочь только с физическим/удалённым доступом к тестовой машине/backup при rehearsal.

WP49 mandatory clean-room portability and RPO/RTO proof

Disaster Recovery rehearsal is simultaneously the Account Independence acceptance test:

1. Start from a fresh approved machine/environment with no original developer working tree.

2. Restore source/release only from accepted GitHub evidence.

3. Run documented bootstrap/deployment.

4. Restore authorized encrypted private backup.

5. Restore runtime secrets through the separate documented secret procedure.

6. Prove installed V10 version/Git SHA.

7. Prove measured recovery meets accepted RPO/RTO.

8. Prove another authorized developer can modify/test Web without receiving OWNER trading credentials.

9. Prove no hidden ChatGPT/website-builder/hosting-dashboard setting is required.

Failure of any point blocks WP50.

WP50 - V10.0.0 Final Release и protected rollback

Цель: Закрыть программу только после доказанного production readiness.

Результаты / deliverables

• Все WP complete; zero open blockers; zero unresolved review threads.

• Final exact-head Regression Shield/security/restore/soak green.

• Owner acceptance Web + Telegram + Guardian + Real safety boundaries.

• Создать V10 protected release/rollback; обновить README/AGENTS/current status.

• V9 остаётся архивным protected recovery/reference, пока owner явно не изменит retention policy.

Обязательная проверка

• Final release manifest/hash verification; clean deploy from release.

Exit Gate

• knowledge/17_V10_CURRENT_PROGRAM_STATUS.md = 100% COMPLETE; дальнейший feature work требует новой программы/maintenance scope.

Зависимости: WP49

Участие владельца: Финальная owner acceptance. Реальная сделка не должна делаться только ради красивой галочки; если нет естественного безопасного сценария, execution acceptance проводится через Sandbox/fake + ранее доказанный Real primitive до отдельного owner решения.

14. Сквозная стратегия тестирования

12. Unit: детерминированные финансовые функции, parsers, validators, state transitions.

13. Contract: API DTO, DB schema, broker adapters, AI schema, RAG/tool registry contracts, Telegram auth.

14. Integration: Core+DB, Core+T-Invest read, Watchtower+Research, Telegram+Web deep-link.

15. Golden regression: V9 fixtures должны защищать от изменения финансового смысла при extraction/refactor.

16. E2E: Web critical paths и Guardian incident lifecycle.

17. Property/fuzz: decimals, identity mapping, malformed source data, replay/idempotency.

18. Security: ASVS/AISVS requirements, OWASP LLM prompt-injection/excessive-agency checks, SAST, dependency, secrets, negative authorization.

19. Failure injection: network/DB/reboot/timeout/duplicate/possible-submit.

20. Shadow evaluation: real portfolio decisions without writes.

21. Production acceptance: только после всех более дешёвых уровней тестирования.

15. Работа с ошибками и инцидентами разработки

Класс	**Пример**	**Правило**	**Переход дальше**
P0 Money/Security	возможен второй submit, утечка trading token	Немедленная остановка relevant program/runtime; kill switch	Запр
P1 Data correctness	неверная валюта/доля/identity/freshness	Исправить в текущем WP + permanent regression	Запрещё
P2 Functional	не работает страница/alert/job	Исправить до Exit Gate, если относится к gate	Только после gate
P3 UI/minor	косметика без влияния на смысл	Можно backlog при документированном решении	Допустим

16. Правила моей будущей работы через GitHub

22. Перед каждым сеансом прочитать AGENTS.md, README.md, PROJECT_MASTER_CONTEXT, V10_CURRENT_PROGRAM_STATUS, этот master-plan, relevant architecture/runbook, open PR/issues и latest Actions.

23. Выбрать ровно один текущий WP из knowledge/17_V10_CURRENT_PROGRAM_STATUS.md. Не начинать соседний WP ради удобства.

24. Создать focused branch; сформулировать acceptance criteria до написания кода.

25. Добавлять tests одновременно с кодом. Если дефект найден - по возможности сначала regression test.

26. Не делать прямых changes в main; branch -> PR -> exact-head gate -> zero unresolved threads -> squash merge.

27. После merge обновить V10_CURRENT_PROGRAM_STATUS отдельным контролируемым изменением или в том же PR, если status отражает реально завершённый gate.

28. Новая идея во время работы попадает в V10_BACKLOG.md, а не реализуется поверх текущего WP без amendment.

29. Любой новый внешний dependency/source сначала проходит official-doc/security/license/maintenance review.

30. Никакой tool/model/service не получает больше authority, чем нужно его узкой функции.

31. Не считать действие выполненным только потому, что код написан: нужна evidence цепочка tests/CI/runtime acceptance.

17. Что намеренно НЕ входит в V10

- Массовая регистрация пользователей и public multi-tenant production.
- Платные подписки/СБП/эквайринг и billing.
- Публичное использование чужих T-Invest API token.
- Public real execution или доверительное управление.
- Мобильные native приложения App Store/Google Play.
- Обучение LLM с нуля и постоянный дорогой GPU cloud без доказанной необходимости.
- Поддержка множества брокеров.
- Маржинальная торговля, short, withdrawal/transfer authority.
- Полная автономная торговля без one-plan/one-owner-approval boundary.

18. Минимальное участие владельца

32. Утвердить V10 master-plan и дать команду начать программу.

33. Через удалённый рабочий стол помочь с первоначальным bootstrap/UAC и фактическим hardware audit.

34. Создать Telegram Bot через BotFather и локально сохранить token, когда Telegram block будет готов.

35. Локально ввести read-only T-Invest token на WP15.

36. После Shadow acceptance и отдельного решения - локально ввести trading token для OWNER execution contour.

37. Предоставить off-device backup место/диск или согласовать альтернативу.

38. Дать owner acceptance перед включением каждого потенциально опасного внешнего/Real режима.

19. Финальный Definition of Done V10

• GitHub является точным source of truth по коду, архитектуре, зависимостям, release и recovery.

• Lenovo автоматически возвращается в HEALTHY/DEGRADED safe state после reboot.

• Core работает без Custom GPT/OpenAI.

• Portfolio/read-only T-Invest слой имеет freshness/completeness и корректную identity.

• Watchtower реально обнаруживает material events, дедуплицирует и создаёт incidents.

• Research хранит provenance; AI failure переводит Copilot/Research в явный degraded mode, но не ломает deterministic Core/Watchtower safety.

• GPT Instructions/Knowledge/Actions migration manifest завершён; System Policy, Knowledge/RAG и Tool Registry имеют versioned source of truth.

• Web AI Copilot и Telegram AI используют один Agent Orchestrator и один authority model.

• Autonomous Research Agent умеет исследовать material events по trusted-source/budget policy без прямого trading authority.

• Custom GPT parity/decommission gate пройден до отказа от платной GPT-зависимости.

• Web полностью выполняет роль основного кабинета и показывает факты/риски/решения/планы/health.

• Telegram доставляет owner alerts и безопасно открывает нужный Web incident.

• Guardian формирует один основной decision либо честный blocker, а target sizing делает Optimizer.

• Shadow Mode пройден на реальном портфеле без unauthorized writes.

• Real execution требует exact immutable plan approval; drift останавливает plan.

• Idempotent replay не создаёт второй submit; ambiguous possible-submit ведёт только к reconciliation.

• Backup реально восстанавливается на fresh environment.

• Fault injection/security/soak tests проходят без P0/P1 blockers.

• Custom GPT можно отключить только после WP33A parity/owner acceptance; Personal Omni-CIO после этого сохраняет conversational AI, мониторинг, research, анализ и Web/Telegram работу, а GPT configuration остаётся архивным recovery/reference asset.

19.1 Final pre-release evidence matrix

Перед V10 = 100% COMPLETE должна существовать evidence matrix, связывающая каждую заявленную capability с:

requirement -> implementation commit -> deterministic tests -> security control -> runtime/shadow evidence -> rollback/recovery evidence -> owner acceptance where required.

Ни сайт, ни презентация, ни AI-текст не являются доказательством capability без этой цепочки. Любая функция, не имеющая evidence chain, считается незавершённой или явно исключённой из release scope.

20. Реестр внешних технических ориентиров и источников

Важно: эти ссылки используются как planning references. Перед каждым implementation WP конкретные версии и требования должны быть повторно проверены по официальным источникам.

• GitHub - Self-hosted runners reference: https://docs.github.com/en/actions/reference/runners/self-hosted-runners - Runner работает на собственной машине, принимает jobs по outbound HTTPS; требует своевременных обновлений.

• GitHub - Adding self-hosted runners: https://docs.github.com/en/actions/how-tos/manage-runners/self-hosted-runners/add-runners - GitHub рекомендует self-hosted runners использовать только с private repositories из-за риска выполнения кода из public forks/PR.

• Telegram - Mini Apps: https://core.telegram.org/bots/webapps - Mini Apps позволяют использовать один mobile-first Web App внутри Telegram; initData требуется валидировать server-side и проверять freshness auth_date.

• OpenAI - GPTs in ChatGPT: https://help.openai.com/en/articles/8554407-gpts-in-chatgpt - Existing GPT capabilities are explicitly separated into Instructions, Knowledge, capabilities and Actions; V10 migration preserves this separation instead of treating the GPT as an opaque model.

• OpenAI - Creating and editing GPTs: https://help.openai.com/en/articles/8554397-creating-a-custom-gpt - Instructions define behavior while Knowledge is reference material; this is the migration basis for System Policy vs Knowledge/RAG.

• OWASP GenAI - LLM01 Prompt Injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/ - AI Copilot/Research security tests must cover prompt injection, excessive agency, improper output handling and retrieval-related trust boundaries.

• Ollama - Tool calling: https://docs.ollama.com/capabilities/tool-calling - Локальный model provider может вызывать ограниченный allowlist tools.

• Ollama - Structured outputs: https://docs.ollama.com/capabilities/structured-outputs - AI output можно принудительно ограничивать JSON schema.

• React - Versions: https://react.dev/versions - На момент формирования документа official docs указывают latest major 19.2.

• Vite - Releases: https://vite.dev/releases - На момент формирования документа current supported line включает Vite 8.1; точный patch фиксировать перед WP2.

• PostgreSQL - Versioning policy: https://www.postgresql.org/support/versioning/ - На момент формирования документа PostgreSQL 18.4 - текущий minor поддерживаемой ветки 18; использовать current supported minor на deployment date.

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/ - Latest stable ASVS 5.0.0 - baseline для Web/API security acceptance.

• OWASP AISVS: https://owasp.org/www-project-artificial-intelligence-security-verification-standard-aisvs-docs/ - AISVS 1.0 released 24.06.2026 - дополнительный security checklist для AI-enabled части.

Technical baseline re-check — 05.08.2026

На этапе финальной проверки плана официальные источники подтверждали следующие ориентиры (версии всё равно перепроверяются перед WP2/WP31 implementation):

• React docs: latest documented line 19.2;

• Vite: 8.1 current supported minor line;

• PostgreSQL: 18.4 current stable release line (PostgreSQL 19 на дату аудита был beta, поэтому не является production baseline V10);

• OWASP ASVS: latest stable 5.0.0;

• OWASP AISVS: version 1.0 live since 24.06.2026;

• Telegram Mini Apps: server-side validation of initData + freshness check via auth_date;

• Ollama: tool calling + JSON-schema structured outputs supported;

• T-Invest: read-only token cannot place orders; account-specific token scoping available;

• GitHub: self-hosted runner uses outbound HTTPS and private-repository use remains the recommended safety posture.

Приложение A. Шаблон V10_CURRENT_PROGRAM_STATUS.md

Рекомендуемая структура канонического файла прогресса:

\# Omni-CIO V10 CURRENT PROGRAM STATUS

Overall: 0% / 100%

Current block: A

Current WP: WP0

State: NOT STARTED

\## Blocks

A Architecture 0/5

B GitHub/CI 0/9

C Lenovo Node 0/12

D Core/Data 0/14

E Web 0/14

F Telegram 0/8

G Research/AI 0/18

H Guardian 0/8

I Real Execution 0/7

J Hardening/Release ... 0/5

\## Current blocker

None

\## Last accepted evidence

• PR: ...

• commit: ...

• Regression Shield: ...

\## Next exact step

WP0 ...

Приложение B. Шаблон PR для каждого WP

- Scope: только один WP/один defect set.
- Why: ссылка на master-plan и current status.
- Financial/safety boundaries: что изменяется и что явно не изменяется.
- Deliverables: список файлов/interfaces/migrations.
- Tests: unit/contract/integration/security/E2E.
- Rollback: как вернуть предыдущий accepted state.
- Owner action: none / exact ручное действие.
- Exit evidence: exact-head SHA, CI run, zero unresolved threads.